Код теста

```
import pytest
import sqlite3
import sys


# ==================== КЛАСС AuthManager ====================

class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
 CREATE TABLE IF NOT EXISTS users
 (
 id
 INTEGER
 PRIMARY
 KEY
 AUTOINCREMENT,
 username
 TEXT
 NOT
 NULL
 UNIQUE,
  password
  TEXT
 NOT
 NULL,
 country
 TEXT
 NOT
  NULL,
 balance
 REAL
 NOT
  NULL
  )
 """)

    def register_user(self, username, password, country, balance):
        with self.connection:
            # Уязвимость: использование format вместо параметризации
            self.connection.execute("""
            INSERT INTO users (username, password, country, balance)
            VALUES ('{}', '{}', '{}', {})
            """.format(username, password, country, balance))

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        # Уязвимость: SQL-инъекция
        cursor.execute("""
        SELECT * FROM users
        WHERE username = '{}' AND password = '{}'
        """.format(username, password))
        return cursor.fetchone()
```

```
    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("DELETE FROM users WHERE id =
{}".format(user_id))

    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("SELECT * FROM users WHERE id = {}".format(user_id))
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("SELECT COUNT(*) FROM users WHERE country =
'{}'".format(country))
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))
            from_balance = cursor.fetchone()[0]

            if from_balance < amount:
                raise ValueError("Insufficient funds")

            self.connection.execute(
                "UPDATE users SET balance = balance - {} WHERE id =
{}".format(amount, from_user_id))
            self.connection.execute("UPDATE users SET balance = balance + {}
WHERE id = {}".format(amount, to_user_id))


# ==================== ФИКСТУРЫ ====================

@pytest.fixture
def db():
    """Фикстура для создания временной базы данных SQLite в памяти."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()


@pytest.fixture
def auth_manager(db):
    """Фикстура для инициализации AuthManager."""
    return AuthManager(db)


# ==================== БАЗОВЫЕ ТЕСТЫ (3 штуки) ====================

@pytest.mark.basic
def test_register_and_get_user(auth_manager):
    """Тест регистрации и получения пользователя по ID."""
    auth_manager.register_user("alice", "pass1", "USA", 100.0)
    user = auth_manager.get_user_by_id(1)
    assert user is not None
    assert user[1] == "alice"  # Проверяем username


@pytest.mark.basic
def test_authenticate_user_success(auth_manager):
    """Тест успешной аутентификации."""
    auth_manager.register_user("bob", "pass2", "UK", 200.0)
```

```python
    user = auth_manager.authenticate_user("bob", "pass2")
    assert user is not None


@pytest.mark.basic
def test_delete_user(auth_manager):
    """Тест удаления пользователя."""
    auth_manager.register_user("charlie", "pass3", "FR", 300.0)
    auth_manager.delete_user(1)
    user = auth_manager.get_user_by_id(1)
    assert user is None


# ==================== ПАРАМЕТРИЗОВАННЫЕ ТЕСТЫ (3 штуки) ====================

@pytest.mark.parametrize("country, expected_count", [
    ("CountryA", 2),
    ("CountryB", 1),
    ("CountryC", 0)
])
def test_count_users_by_country(auth_manager, country, expected_count):
    """Параметризованный тест подсчета пользователей по странам."""
    auth_manager.register_user("u1", "p", "CountryA", 100)
    auth_manager.register_user("u2", "p", "CountryA", 100)
    auth_manager.register_user("u3", "p", "CountryB", 100)

    count = auth_manager.count_users_by_country(country)
    assert count == expected_count


@pytest.mark.parametrize("username, password, should_auth", [
    ("user1", "correct_pass", True),
    ("user1", "wrong_pass", False),
    ("nonexistent", "correct_pass", False)
])
def test_authenticate_various_credentials(auth_manager, username, password,
should_auth):
    """Параметризованный тест аутентификации с разными данными."""
    auth_manager.register_user("user1", "correct_pass", "US", 100)
    result = auth_manager.authenticate_user(username, password)

    if should_auth:
        assert result is not None
    else:
        assert result is None


@pytest.mark.parametrize("transfer_amount, expected_sender_bal,
expected_receiver_bal", [
    (100, 900.0, 600.0),
    (500, 500.0, 1000.0),
    (0, 1000.0, 500.0)
])
def test_transfer_balance_amounts(auth_manager, transfer_amount,
expected_sender_bal, expected_receiver_bal):
    """Параметризованный тест перевода средств с разными суммами."""
    auth_manager.register_user("sender", "p", "US", 1000)
    auth_manager.register_user("receiver", "p", "US", 500)

    auth_manager.transfer_balance(1, 2, transfer_amount)

    assert auth_manager.get_user_by_id(1)[4] == expected_sender_bal
    assert auth_manager.get_user_by_id(2)[4] == expected_receiver_bal
```

```python
# ==================== ТЕСТЫ ИСКЛЮЧЕНИЙ (2 штуки) ====================

@pytest.mark.exception
def test_transfer_insufficient_funds(auth_manager):
    """Тест исключения при недостатке средств."""
    auth_manager.register_user("poor", "p", "US", 100)
    auth_manager.register_user("rich", "p", "US", 1000)

    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 200)


@pytest.mark.exception
def test_register_duplicate_username(auth_manager):
    """Тест исключения при регистрации дубликата username."""
    auth_manager.register_user("duplicate", "p", "US", 100)

    with pytest.raises(sqlite3.IntegrityError):
        auth_manager.register_user("duplicate", "p2", "US", 200)


# ==================== ТЕСТ НА SQL-ИНЪЕКЦИЮ ====================

@pytest.mark.security
def test_sql_injection_auth_bypass(auth_manager):
    """
    SQL-инъекция в authenticate_user - обход аутентификации.
    Злоумышленник может войти как любой пользователь без пароля.
    """
    auth_manager.register_user("admin", "supersecret", "US", 1000)

    # Классическая SQL-инъекция для обхода проверки пароля
    malicious_password = "' OR '1'='1"
    user = auth_manager.authenticate_user("admin", malicious_password)

    assert user is not None, "SQL Injection: auth bypass успешен!"
    assert user[1] == "admin"


@pytest.mark.security
def test_sql_injection_register_union_attack(auth_manager):
    """
    SQL-инъекция в register_user - UNION атака для извлечения данных.
    """
    auth_manager.register_user("victim", "real_password", "US", 1000)

    # Пытаемся внедрить дополнительные данные через UNION
    malicious_username = "hacker'; INSERT INTO users (username, password,
country, balance) VALUES ('backdoor', 'hacked', 'RU', 9999); --"

    try:
        auth_manager.register_user(malicious_username, "pass", "US", 100)
    except:
        pass  # Ожидаем ошибку или успешное выполнение

    # Проверяем, не появился ли "backdoor" пользователь
    cursor = auth_manager.connection.cursor()
    cursor.execute("SELECT COUNT(*) FROM users WHERE username='backdoor';")
    count = cursor.fetchone()[0]

    # Если count > 0, значит инъекция сработала и создала пользователя!
    # Это демонстрирует критическую уязвимость
    assert count >= 0
```

```python
@pytest.mark.security
def test_sql_injection_get_user_by_id(auth_manager):
    """
    SQL-инъекция в get_user_by_id - извлечение всех пользователей.
    """
    auth_manager.register_user("user1", "pass1", "US", 100)
    auth_manager.register_user("user2", "pass2", "UK", 200)

    # Вместо ID подставляем SQL-инъекцию
    malicious_id = "1 OR 1=1"
    user = auth_manager.get_user_by_id(malicious_id)

    # Из-за инъекции "SELECT * FROM users WHERE id = 1 OR 1=1"
    # вернётся ПЕРВЫЙ пользователь, но условие 1=1 истинно для всех
    assert user is not None


@pytest.mark.security
def test_sql_injection_delete_users(auth_manager):
    """
    SQL-инъекция в delete_user - массовое удаление пользователей.
    """
    auth_manager.register_user("user1", "pass1", "US", 100)
    auth_manager.register_user("user2", "pass2", "UK", 200)
    auth_manager.register_user("user3", "pass3", "FR", 300)

    # Проверяем, что есть 3 пользователя
    cursor = auth_manager.connection.cursor()
    cursor.execute("SELECT COUNT(*) FROM users;")
    count_before = cursor.fetchone()[0]
    assert count_before == 3

    # Пытаемся удалить ВСЕХ пользователей через инъекцию
    malicious_id = "1; DELETE FROM users; --"
    try:
        auth_manager.delete_user(malicious_id)
    except:
        pass  # SQLite может не выполнить несколько команд

    # В реальной БД с PostgreSQL/MySQL все пользователи были бы удалены!
    # Этот тест показывает потенциальную уязвимость


@pytest.mark.security
def test_sql_injection_count_by_country(auth_manager):
    """
    SQL-инъекция в count_users_by_country - манипуляция с подсчётом.
    """
    auth_manager.register_user("user1", "pass1", "US", 100)
    auth_manager.register_user("user2", "pass2", "US", 200)
    auth_manager.register_user("user3", "pass3", "UK", 300)

    # Нормальный запрос
    normal_count = auth_manager.count_users_by_country("US")
    assert normal_count == 2

    # Инъекция: пытаемся получить количество ВСЕХ пользователей
    malicious_country = "' OR '1'='1"
    injected_count = auth_manager.count_users_by_country(malicious_country)

    # Запрос стал: SELECT COUNT(*) FROM users WHERE country = '' OR '1'='1'
    # Это вернёт общее количество пользователей (3)
```

```
    assert injected_count == 3, "SQL Injection: получено общее количество
пользователей!"

@pytest.mark.security
def test_sql_injection_stack_queries(auth_manager):
    """
    SQL-инъекция с несколькими командами (stacked queries).
    """
    auth_manager.register_user("victim", "pass", "US", 100)

    # Пытаемся выполнить несколько команд через точку с запятой
    malicious_username = "attacker'; UPDATE users SET balance = 0 WHERE
username='victim'; --"

    try:
        auth_manager.register_user(malicious_username, "pass", "US", 100)
    except:
        pass

    # Проверяем баланс жертвы
    cursor = auth_manager.connection.cursor()
    cursor.execute("SELECT balance FROM users WHERE username='victim';")
    result = cursor.fetchone()

    if result:
        balance = result[0]
        # Если balance == 0, значит инъекция сработала и обнулила счёт!
        # assert balance != 0  # Раскомментируйте для проверки уязвимости


@pytest.mark.security
def test_sql_injection_extract_passwords(auth_manager):
    """
    SQL-инъекция для извлечения хешей паролей (теоретическая атака).
    Показывает, что можно получить доступ к конфиденциальным данным.
    """
    auth_manager.register_user("user1", "secret123", "US", 100)
    auth_manager.register_user("user2", "password456", "UK", 200)

    # Атака: пытаемся получить всех пользователей, игнорируя пароль
    malicious_input = "' OR '1'='1"
    users = auth_manager.authenticate_user("anyuser", malicious_input)

    # Получили первого пользователя из БД
    assert users is not None
    # В реальности можно извлечь ВСЕ пароли перебором или UNION атакой


# ==================== ПРОПУСКАЕМЫЙ ТЕСТ ====================

@pytest.mark.skip(reason="Тест временно пропущен: функционал 2FA не
реализован в данной версии")
def test_two_factor_authentication():
    """Пропускаемый тест для демонстрации метки skip."""
    assert False


# ==================== ТЕСТ ДЛЯ ДЕМОСТРАЦИИ ОШИБКИ (НЕПРАВИЛЬНЫЙ РЕЗУЛЬТАТ)
====================

def test_intentional_fail_for_screenshot():
    """Этот тест намеренно падает, чтобы сделать скриншот 'Неправильного
результата'."""
    assert 1 == 2, "Намеренная ошибка для скриншота"
```

Правильный результат

```
(.venv) PS C:\Users\ofits\PycharmProjects\WelcomeScreen> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ofits\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ofits\PycharmProjects\WelcomeScreen
configfile: pytest.ini
collected 22 items

test_auth_manager.py::test_register_and_get_user PASSED                  [  4%]
test_auth_manager.py::test_authenticate_user_success PASSED              [  9%]
test_auth_manager.py::test_delete_user PASSED                            [ 13%]
test_auth_manager.py::test_count_users_by_country[CountryA-2] PASSED     [ 18%]
test_auth_manager.py::test_count_users_by_country[CountryB-1] PASSED     [ 22%]
test_auth_manager.py::test_count_users_by_country[CountryC-0] PASSED     [ 27%]
test_auth_manager.py::test_authenticate_various_credentials[user1-correct_pass-True] PASSED [ 31%]
test_auth_manager.py::test_authenticate_various_credentials[user1-wrong_pass-False] PASSED [ 36%]
test_auth_manager.py::test_authenticate_various_credentials[nonexistent-correct_pass-False] PASSED [ 40%]
test_auth_manager.py::test_transfer_balance_amounts[100-900.0-600.0] PASSED [ 45%]
test_auth_manager.py::test_transfer_balance_amounts[500-500.0-1000.0] PASSED [ 50%]
test_auth_manager.py::test_transfer_balance_amounts[0-1000.0-500.0] PASSED [ 54%]
test_auth_manager.py::test_transfer_insufficient_funds PASSED            [ 59%]
test_auth_manager.py::test_register_duplicate_username PASSED            [ 63%]
test_auth_manager.py::test_sql_injection_auth_bypass PASSED              [ 68%]
test_auth_manager.py::test_sql_injection_register_union_attack PASSED    [ 72%]
test_auth_manager.py::test_sql_injection_get_user_by_id PASSED           [ 77%]
test_auth_manager.py::test_sql_injection_delete_users PASSED             [ 81%]
test_auth_manager.py::test_sql_injection_count_by_country PASSED         [ 86%]
test_auth_manager.py::test_sql_injection_stack_queries PASSED            [ 90%]
test_auth_manager.py::test_sql_injection_extract_passwords PASSED        [ 95%]
test_auth_manager.py::test_two_factor_authentication SKIPPED (Тест временно пропущен: функционал 2FA не реализован в данной версии) [100%]

======================== 21 passed, 1 skipped in 0.07s =========================
```

Ошибочный результат

```
(.venv) PS C:\Users\ofits\PycharmProjects\WelcomeScreen> pytest -v --maxfail=2 --tb=short
============================= test session starts ==============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ofits\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ofits\PycharmProjects\WelcomeScreen
configfile: pytest.ini
collected 23 items

test_auth_manager.py::test_register_and_get_user PASSED                  [  4%]
test_auth_manager.py::test_authenticate_user_success PASSED              [  8%]
test_auth_manager.py::test_delete_user PASSED                            [ 13%]
test_auth_manager.py::test_count_users_by_country[CountryA-2] PASSED     [ 17%]
test_auth_manager.py::test_count_users_by_country[CountryB-1] PASSED     [ 21%]
test_auth_manager.py::test_count_users_by_country[CountryC-0] PASSED     [ 26%]
test_auth_manager.py::test_authenticate_various_credentials[user1-correct_pass-True] PASSED [ 30%]
test_auth_manager.py::test_authenticate_various_credentials[user1-wrong_pass-False] PASSED [ 34%]
test_auth_manager.py::test_authenticate_various_credentials[nonexistent-correct_pass-False] PASSED [ 39%]
test_auth_manager.py::test_transfer_balance_amounts[100-900.0-600.0] PASSED [ 43%]
test_auth_manager.py::test_transfer_balance_amounts[500-500.0-1000.0] PASSED [ 47%]
test_auth_manager.py::test_transfer_balance_amounts[0-1000.0-500.0] PASSED [ 52%]
test_auth_manager.py::test_transfer_insufficient_funds PASSED            [ 56%]
test_auth_manager.py::test_register_duplicate_username PASSED            [ 60%]
test_auth_manager.py::test_sql_injection_auth_bypass PASSED              [ 65%]
test_auth_manager.py::test_sql_injection_register_union_attack PASSED    [ 69%]
test_auth_manager.py::test_sql_injection_get_user_by_id PASSED           [ 73%]
test_auth_manager.py::test_sql_injection_delete_users PASSED             [ 78%]
test_auth_manager.py::test_sql_injection_count_by_country PASSED         [ 82%]
test_auth_manager.py::test_sql_injection_stack_queries PASSED            [ 86%]
test_auth_manager.py::test_sql_injection_extract_passwords PASSED        [ 91%]
test_auth_manager.py::test_two_factor_authentication SKIPPED (Тест временно пропущен: функционал 2FA не реализован в данной версии) [ 95%]
test_auth_manager.py::test_intentional_fail_for_screenshot FAILED        [100%]

================================== FAILURES ===================================
_____________________ test_intentional_fail_for_screenshot _____________________
test_auth_manager.py:365: in test_intentional_fail_for_screenshot
    assert 1 == 2, "Намеренная ошибка для скриншота"
E   AssertionError: Намеренная ошибка для скриншота
E   assert 1 == 2
=========================== short test summary info ===========================
FAILED test_auth_manager.py::test_intentional_fail_for_screenshot - AssertionError: Намеренная ошибка для скриншота
======================== 1 failed, 21 passed, 1 skipped in 0.16s ========================
```